

SMC Report Engine - How To

SMC Report Engine — How To

Location: `~/ZDST-TEST-2/SMC/` on the NG-central JupyterHub (<https://bauchi-hcm.digit.org/jupyter/user/reportsadmin/>)

This engine generates SMC campaign reports for all 10 Nigerian states from one shared set of scripts. You pick a state, tick the reports you want, choose either a cycle or a date range, and press a button. There is no per-state copy of anything.

```
SMC/
├─ START_HERE_SMC.ipynb          <- the only file a report user opens
├─ _engine/
│   ├── report_ui.py             <- the form, progress, Stop button, download links
│   ├── campaign-config.json     <- which states exist, and which reports each one gets
│   ├── reports_config.json      <- the report catalog (script path, output name, filters)
│   ├── reports_date_config.json <- written automatically on every run; do not hand-edit
│   ├── COMMON_UTILS/
│   │   ├── common_utils.py      <- get_resp (ES calls + cycle injection), save_excel
│   │   ├── tenant.py            <- PREFIX / CAMPAIGN / es_index() for the selected state
│   │   └─ custom_date_utils.py  <- reads the reporting window
│   └─ REPORTS_SCRIPTS/
│       ├── children_treated/child_treatment_report.py
│       ├── ri_report/ri_report.py
│       ├── ors_report/ors_report.py
│       └─ stock_report/stock_report.py
└─ FINAL_REPORTS/<state>/      <- generated workbooks land here
```

1. How to generate reports

Step 1 — open the notebook

In the JupyterLab file browser open `ZDST-TEST-2/SMC/START_HERE_SMC.ipynb` .

Step 2 — run the single cell

Click the cell and press **Shift+Enter**. That is the only cell, and its only job is to launch the form:

```
import os, sys, importlib
sys.path.insert(0, os.path.join(os.getcwd(), '_engine'))
import report_ui; importlib.reload(report_ui)
report_ui.launch(debug=False)  # set debug=True for full logs
```

If nothing appears, the kernel is not connected yet — wait for the kernel indicator to go idle and run the cell again.

The screenshot shows the 'SMC Report Generator' web interface. At the top, it says 'Pick one state, tick its report(s), choose cycle(s) or a date range, then Generate report.' Under the 'State' section, there are radio buttons for various Nigerian states, with 'Sokoto (so)' selected. The 'Reports' section has a 'Select All Reports' checkbox and four individual checkboxes: 'Child Treated Report' (checked), 'Stock Report' (unchecked), 'ORS/ORZ Report' (checked), and 'RI Report' (checked). The 'Reporting period' section has a 'Period:' label and two radio buttons: 'By cycle' (selected) and 'Custom date range'. Below this is a 'Load cycles' button and a message: 'Loaded 3 cycle(s) for Child Treated Report, ORS/ORZ Report, RI Report: 01, 02, 03'. There are three checkboxes for cycles: 'Cycle 01 (2026-07-22 -- 2026-08-05)' (checked), 'Cycle 02 (2026-08-19 -- 2026-09-03)' (unchecked), and 'Cycle 03 (2026-08-22 -- 2026-09-18)' (unchecked). A 'Debug (full logs)' checkbox is also present. At the bottom, there are two buttons: 'Generate report' (green) and 'Stop report generation' (red). A 'Progress' bar is at the very bottom.

Step 3 — pick one state

This close-up shows the 'State' section of the interface. It includes the label 'State:' and a row of radio buttons for the following states: Oyo (oy), Kogi (ko), Nasarawa (na), FCT Abuja (ab), Bauchi (ba), Borno (bo), Kebbi (ke), Plateau (pl), Sokoto (so), and Zamfara (zf). The 'Sokoto (so)' radio button is selected.

One state per run. The tenant prefix in brackets (ba , zf , so ...) is the Elasticsearch index prefix that the scripts will read.

Step 4 — tick the reports you want

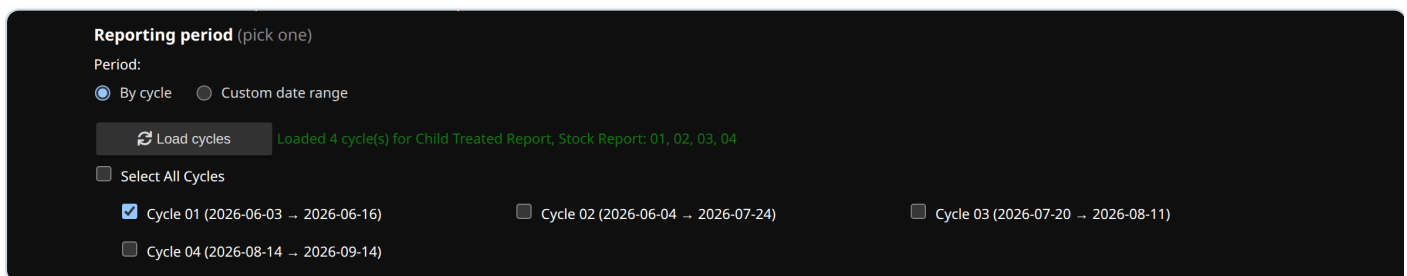
This close-up shows the 'Reports' section of the interface. It includes the label 'Reports (for the selected state)' and a 'Select All Reports' checkbox. Below it are four checkboxes: 'Child Treated Report' (checked), 'Stock Report' (unchecked), 'ORS/ORZ Report' (checked), and 'RI Report' (checked).

Only the reports available to the selected state are listed. **Child Treated** and **Stock** are available everywhere; **RI** and **ORS/ORZ** appear only for the states that ran them. See §4 for how that is decided.

Step 5 — choose the period

Two modes:

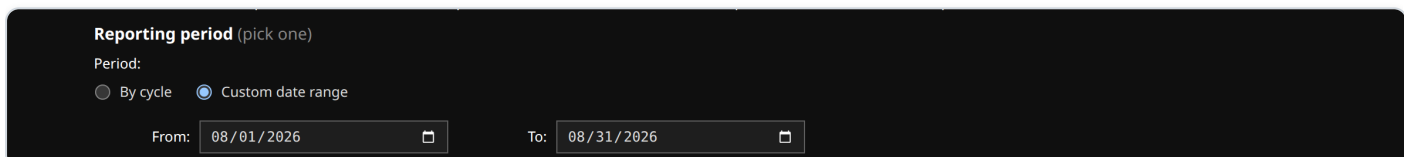
By cycle (recommended). Press **Load cycles**. The engine queries live data and shows only the cycles that actually contain data *for the reports you ticked*, with each cycle's real date span. Tick one or more.



The screenshot shows a dark-themed interface for selecting a reporting period. At the top, it says 'Reporting period (pick one)'. Below that, 'Period:' is followed by two radio buttons: 'By cycle' (which is selected) and 'Custom date range'. A 'Load cycles' button with a refresh icon is visible. To its right, a green message states 'Loaded 4 cycle(s) for Child Treated Report, Stock Report: 01, 02, 03, 04'. Below this, there is a 'Select All Cycles' checkbox. Underneath, four cycle options are listed, each with a checkbox and a date range: 'Cycle 01 (2026-06-03 → 2026-06-16)' (checked), 'Cycle 02 (2026-06-04 → 2026-07-24)' (unchecked), 'Cycle 03 (2026-07-20 → 2026-08-11)' (unchecked), and 'Cycle 04 (2026-08-14 → 2026-09-14)' (unchecked).

Selecting several cycles reports across the whole combined span.

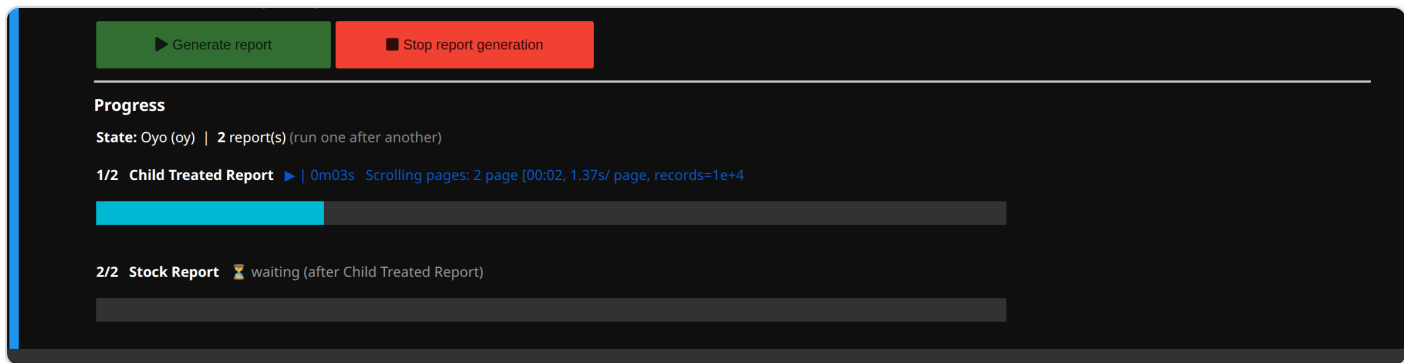
Custom date range. Switch the Period radio to *Custom date range* and pick From/To.



The screenshot shows the same dark-themed interface, but the 'Custom date range' radio button is now selected. The 'Load cycles' button is no longer present. Instead, there are two date input fields: 'From:' with the value '08/01/2026' and 'To:' with the value '08/31/2026'. Both fields have a calendar icon to their right.

In cycle mode the date window is deliberately ignored (the engine writes a wide-open 2000–2099 window so the cycle alone scopes the data). In date mode there is no cycle filter and the dates apply. Use one or the other, not both.

Step 6 — Generate report



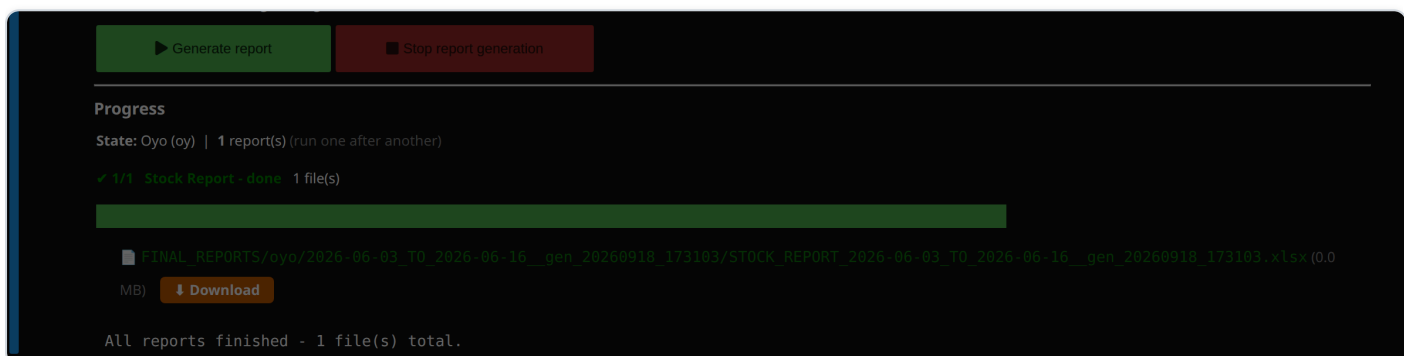
Progress is shown per report, with the elapsed time and the current [Step x/y] . Reports run **one after another**, not in parallel — the queued ones show 🕒 *waiting*.

Once fetching hits 100% the Excel write stage begins, and on a large report that stage alone can take a few minutes. It is not stuck: the label switches to a row counter, e.g.

```
[save_excel] 50,000/256,830 rows written (19%) -> CHILDREN_TREATED.xlsx
```

Stop report generation terminates the running report. Partial output is deleted, so you never receive a truncated workbook.

Step 7 — download



Each finished file is listed with its size and an orange ↓ **Download** button. You can also find the files in `SMC/FINAL_REPORTS/<state>/` .

Reports over **1,000,000 rows** are split automatically into `<name>_part1.xlsx` , `<name>_part2.xlsx` , ... Excel cannot hold more than ~1,048,576 rows in one sheet.

2. How to add a new report — common to all states

Two files to touch, plus the script itself. Worked example: the Stock report, added this way.

Step 1 — write the script

Create `_engine/REPORTS_SCRIPTS/<your_report>/<your_report>.py` . Follow §5 — the rules there are what make one script work for ten states.

Step 2 — register it in the catalog

`_engine/reports_config.json` :

```
{
  "Stock Report": {
    "input":    "/REPORTS_SCRIPTS/stock_report/",
    "scripts":  ["stock_report.py"],
    "filename": ["STOCK_REPORT.xlsx"],
    "cycle_index": "stock"
  }
}
```

Key	Meaning
<code>input</code>	folder holding the script, relative to <code>_engine</code>
<code>scripts</code>	script(s) to run, in order
<code>filename</code>	workbook name(s) the UI should look for and offer as a download
<code>cycle_index</code>	<i>optional</i> — which ES index to discover cycles in. Defaults to <code>project-task</code> . Set it when your report reads a different index (Stock reads <code>stock-index</code>)
<code>cycle_filter</code>	<i>optional</i> — see §4

Step 3 — make it common

`_engine/campaign-config.json` :

```
{
  "campaign_type": "SMC",
  "common_reports": ["Child Treated Report", "Stock Report"],
  "states": { "...": "..." }
}
```

Anything in `common_reports` is offered for **every** state. That is the whole change — no per-state edits.

Step 4 — verify before telling anyone

```
cd ~/ZDST-TEST-2/SMC/_engine
python3 -c "import
py_compile;py_compile.compile('REPORTS_SCRIPTS/stock_report/stock_report.py',doraise=True)"

# run it exactly as the UI does: state via env, cycle via env
STATE_CODE=zamfara CYCLES=02 python3 REPORTS_SCRIPTS/stock_report/stock_report.py
```

Then run it for a second state to prove nothing is state-specific, and confirm two different cycles give two different results (if they don't, your cycle filter isn't working — see §5.2).

3. How to add a new report — specific states only

Same script and same catalog entry as §2. The only difference: instead of adding it to `common_reports`, list it under the states that should have it.

`_engine/campaign-config.json`:

```

{
  "common_reports": ["Child Treated Report", "Stock Report"],
  "states": {
    "kogi": {
      "name": "Kogi",
      "tenant_id": "ko",
      "campaign_number": "CMP-2026-06-03-000315",
      "extra_reports": ["RI Report"]
    },
    "sokoto": {
      "name": "Sokoto",
      "tenant_id": "so",
      "campaign_number": "CMP-2026-07-22-000449",
      "extra_reports": ["ORS/ORZ Report", "RI Report"]
    },
    "oyo": {
      "name": "Oyo",
      "tenant_id": "oy",
      "campaign_number": "CMP-2026-06-03-000308"
    }
  }
}

```

A state's report list is `common_reports` + its own `extra_reports`. Oyo has no `extra_reports`, so it sees only the two common ones.

The name in `extra_reports` **must match the catalog key exactly**, including spaces and punctuation — `"ORS/ORZ Report"`, NOT `"ORS Report"`.

4. How to add a new report — specific states, specific cycles only

This one works differently from §2 and §3, and the difference matters.

Cycles are never listed in config. They are discovered from live data every time the user presses *Load cycles*. This is deliberate: a hardcoded cycle in the old per-state RI script was silently discarding 79% / 73% / 86% of Kogi / Plateau / Sokoto's RI records, because those states carry RI data in different cycles.

You restrict a report to certain cycles by declaring **what its data looks like**, using `cycle_filter` in `reports_config.json`. The engine then shows only the cycles where that filter actually matches data, per state.

```
{
  "RI Report": {
    "input":    "/REPORTS_SCRIPTS/ri_report/",
    "scripts":  ["ri_report.py"],
    "filename": ["RI_Report.xlsx"],
    "cycle_filter": { "term": { "Data.additionalDetails.flow.keyword": "riDone" } }
  },
  "ORS/ORZ Report": {
    "input":    "/REPORTS_SCRIPTS/ors_report/",
    "scripts":  ["ors_report.py"],
    "filename": ["ORSZ_TREATED.xlsx"],
    "cycle_filter": { "term": { "Data.productName.keyword": "ORS-Zinc" } }
  }
}
```

`cycle_filter` may be a single clause or a list of clauses; they are ANDed into the discovery query's `must`.

What that produces today

Live, as configured right now — note how the same report offers different cycles in different states, with no static configuration anywhere:

state	report	cycles offered
oyo	Child Treated, Stock	01, 02, 03, 04
kogi	Child Treated, Stock	01, 02, 03, 04
kogi	RI	02, 03, 04 — no RI data in cycle 01
nasarawa	Child Treated, Stock	01, 02, 03, 04
fct_abuja	Child Treated, Stock	01, 02, 03, 04
bauchi	Child Treated, Stock	01, 02, 03, 04
borno	Child Treated, Stock	01, 02, 03
kebbi	Child Treated, Stock	01, 02, 03
kebbi	ORS/ORZ	01, 03 — no ORS-Zinc in cycle 02
plateau	Child Treated, Stock	01, 02, 03, 04
plateau	RI	01, 02, 03, 04
sokoto	Child Treated, Stock	01, 02, 03
sokoto	ORS/ORZ	01, 03
sokoto	RI	02, 03
zamfara	Child Treated, Stock	01, 02, 03

A cycle is listed only if it holds at least `CYCLE_MIN_DOCS` (100) matching documents, which keeps stray test records out of the list.

If you need a hard cycle restriction

There is currently **no** config key that pins a report to an explicit cycle list for a given state — and in most cases you should not want one, because `cycle_filter` already produces the correct answer from the data. If you have a genuine case (for example a cycle whose data is known-bad and must not be reportable), that needs a small engine change in `report_ui._load_cycles` — roughly:

```
"My Report": { "cycle_allow": { "sokoto": ["02", "03"] } }
```

Raise it as a change request rather than hardcoding a cycle inside a report script. A cycle hardcoded in a script cannot be seen from the UI and is exactly how the RI data-loss bug happened.

5. Script conventions

Every report script must follow these, or it will work for one state and quietly break for the other nine.

5.1 Never hardcode tenant or campaign

```
from COMMON_UTILS.tenant import PREFIX, CAMPAIGN, es_index

CAMPAIGN_NUMBER = CAMPAIGN          # resolved from the selected state
ES_STOCK_INDEX = es_index("stock") # -> https://.../<prefix>-stock-index-v1/_search
```

`tenant.py` reads `STATE_CODE` from the environment (the UI sets it) and looks the state up in `campaign-config.json`. It raises if the state is not configured, so a typo fails loudly.

5.2 Never hardcode the cycle — and never keep your own cycle clause

`get_resp` injects the user's cycle selection into your query, but **only if the query has no cycle clause of its own**:

```
present = any(key in m.get("terms", {}) or key in m.get("term", {}) for m in arr)
if not present:
    arr.append({"terms": {key: cycle_terms(cyc)}})
```

So a leftover `{"term": {"Data.additionalDetails.cycleIndex.keyword": "02"}}` does not merely duplicate the filter — it **suppresses injection entirely** and pins every run to cycle 02 regardless of what the user ticked. Leave the cycle out of your query completely.

Injection only applies to fact indexes:

```
CYCLE_INDEXES = ("project-task-index", "hf-referral-index", "referral-index",
                 "stock-index", "crosscycle-index")
```

Lookup indexes (`individual`, `household-member`, `household`, `project-beneficiary`) are deliberately excluded — filtering them by cycle breaks the `clientId` join and blanks out Child Name / Household Head Name.

Your query must be a dict with `query.bool.must` (or `.filter`) as a **list**, or injection cannot find a place to add the clause.

5.3 Take the reporting window from the shared helper

```
from COMMON_UTILS.custom_date_utils import get_custom_dates_of_reports
lteTime, gteTime, start_date_str, end_date_str = get_custom_dates_of_reports()
...
{"range": {"Data.@timestamp": {"gte": gteTime, "lte": lteTime}}}
```

Use `Data.@timestamp` — that is what all four existing reports use. In cycle mode this is a no-op; in date mode it is the filter.

5.4 Write output only through `save_excel`

```
from COMMON_UTILS.common_utils import save_excel
save_excel(df, out_path) # single sheet
save_excel(df, out_path, extra_sheets={"Summary": sdf}) # extra sheets go in file 1
```

This gives you `xlswriter` `constant_memory` streaming, automatic >1M-row part-file splitting, and a loud failure if no Excel engine is present. **Do not** use `df.to_excel`, `pd.ExcelWriter`, `to_csv`, or `openpyxl` directly — you would lose the row-limit protection and silently change the delivered format.

5.5 Put every tunable in one CONFIG block

One `# ===== CONFIG =====` block near the top: campaign, index URLs, page sizes, filter value lists, output directory and filename. Define the ES host **once**. The next person should be able to retune the report without reading the body.

5.6 Print `[Step x/y]` markers

```
log("[Step 1/5] Fetching State Facility -> HF (ISSUED) ...")
```

The UI parses `Step n/m` to drive its progress bar and to keep the label honest during long silent stages.

5.7 Resolve paths from `__file__`, never the working directory

```
file_path = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
OUTPUT_DIR = os.path.join(file_path, "FINAL_REPORTS", "STOCK")
```

5.8 Fail loudly, never silently empty

If a query returns nothing, say so. A report that writes a valid but empty workbook without comment looks identical to a successful run.

6. Config reference

campaign-config.json

```
{
  "campaign_type": "SMC",
  "common_reports": ["Child Treated Report", "Stock Report"],
  "states": {
    "<state_code>": {
      "name": "Display Name",
      "tenant_id": "xx",
      "campaign_number": "CMP-...",
      "extra_reports": ["Optional Report"]
    }
  }
}
```

reports_config.json

```
{
  "<Report Name>": {
    "input": "/REPORTS_SCRIPTS/<folder>",
    "scripts": ["<script>.py"],
    "filename": ["<OUTPUT>.xlsx"],
    "cycle_index": "stock",
    "cycle_filter": { "term": { "<field>.keyword": "<value>" } }
  }
}
```

Environment the UI sets for each script

Variable	Example	Meaning
STATE_CODE	zamfara	selected state; <code>tenant.py</code> resolves prefix + campaign
CYCLES	01,02	ticked cycles, empty in date mode

`reports_date_config.json`

Rewritten automatically before every run — **do not hand-edit it**. In cycle mode it gets `2000-01-01 ... 2099-12-31` so the date filter is inert; in date mode it gets the picked dates. It is the handoff channel between the UI and the scripts, not a settings file.

7. Troubleshooting

Symptom	Cause / fix
Form doesn't appear after Shift+Enter	Kernel not connected. Wait for idle, re-run the cell.
"Load cycles" shows no cycles	The report's <code>cycle_filter</code> matches nothing in that state, or <code>cycle_index</code> points at the wrong index. Check the filter against live data first.
Progress sits at 100% for minutes	Normal — the Excel write runs after the fetch completes. Watch for the <code>[save_excel] n/N rows written</code> counter to confirm it is moving.
A new report always returns the same numbers whatever cycle is ticked	A cycle clause is still in the script's query, suppressing injection. See §5.2.
<code>STATE_CODE 'x'</code> not configured	The state is missing from <code>campaign-config.json</code> .
Report not offered for a state	Not in <code>common_reports</code> and not in that state's <code>extra_reports</code> ; or the catalog key doesn't match character-for-character.
Output missing / truncated	If a run was stopped, partial output is deleted by design. Re-run it.
The file list shows a report you didn't run, or the file count is higher than the number of files on disk	After each report the engine sweeps everything in <code>_engine/FINAL_REPORTS</code> into the run folder, so a stale file left there by a crashed run (or by running a script by hand from the command line) gets attributed to whichever report finishes first. Check <code>_engine/FINAL_REPORTS</code> is empty before a run; it is staging, not storage.
Row count near 1,048,576	Look for <code>_part1</code> / <code>_part2</code> files — the split is automatic.

Appendix — verifying a change without the UI

```
cd ~/ZDST-TEST-2/SMC/_engine

# compile everything
python3 - <<'EOF'
import glob, py_compile
for p in glob.glob('**/*.py', recursive=True):
    py_compile.compile(p, doraise=True)
print("all modules compile")
EOF

# cycle mode, one state, one cycle
STATE_CODE=zamfara CYCLES=02 python3 REPORTS_SCRIPTS/stock_report/stock_report.py

# prove the cycle filter is live: these must differ
STATE_CODE=zamfara CYCLES=01 python3 REPORTS_SCRIPTS/stock_report/stock_report.py
STATE_CODE=zamfara CYCLES=03 python3 REPORTS_SCRIPTS/stock_report/stock_report.py
```

Always test a second state before shipping. One state passing proves nothing about the per-tenant wiring.